


# 密码分析学实验报告

姓名：邱泽辉、江李阳、王健一

学号：202400460046、202400460042、202400460037

专业：密码科学与技术

日期：October 20, 2026

# Contents

**1 实验环境** **2**

**2 实验原理** **2**
2.1 线性近似表达式 . . . . . 2
2.1.1 线性近似表达式的概率 . . . . . 2
2.2 线性路线 . . . . . 2
2.3 线性近似表 . . . . . 2

**3 实验内容** **3**
3.1 任务一 . . . . . 3
3.2 任务二 . . . . . 4
3.2.1 考量: 活跃 S 盒的个数 . . . . . 4
3.2.2 贪心算法 . . . . . 6
3.2.3 意外之喜 . . . . . 8
3.3 任务三 . . . . . 11
3.3.1 $\alpha_L \neq \beta_R$ . . . . . 12
3.3.2 $\alpha_L = \beta_R$ . . . . . 12

**4 实验报告完整地址** **15**

**5 AI 使用情况** **15**

# 1 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约为 2.1 GHz；内存 16 GB。
- 软件环境：Windows、MacOS,Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。

# 2 实验原理

## 2.1 线性近似表达式

针对密码算法 $E_K(\cdot)$，对于一对给定的线性掩码 $(\alpha, \beta)$ 可视为 $E_K(\cdot)$ 的一个线性近似，并称

$$\alpha \cdot x \oplus \beta \cdot E_K(x)$$

为 $E_K(\cdot)$ 的一个线性近似表达式，其中 $\alpha$ 和 $\beta$ 分别被称为线性近似的输入掩码和输出掩码。

### 2.1.1 线性近似表达式的概率

依据上述对线性近似表达式的定义，给定一个线性近似 $(\alpha, \beta)$，相应的线性近似表达式的概率 $Pr(\alpha, \beta)$ 为使得线性近似表达式取 0 的输入在整个输入空间中的占比，对应如下计算式子：

$$Pr(\alpha, \beta) = \frac{\#\{X \in \mathbb{F}_2^n | \alpha \cdot X \oplus \beta \cdot E_K(X) = 0\}}{2^n}$$

其中定义线性近似的偏差为线性近似表达式的概率与 $1/2$ 的差值，即

$$\varepsilon(\alpha, \beta) = Pr(\alpha, \beta) - \frac{1}{2}$$

## 2.2 线性路线

对于迭代型分组密码算法 $E_K(\cdot)$，设输入和输出掩码分别为 $\alpha$、$\beta$，中间状态为 $Y_1, Y_2, \cdots, Y_{r-1}$ 处掩码分别为 $\alpha_1, \alpha_2, \cdots, \alpha_{r-1}$，则称 $(\alpha_1, \alpha_2, \cdots, \alpha_{r-1}, \beta)$ 为 $E_K(\cdot)$ 的一条线性路线。注意到这里线性近似与线性路线是不同的概念，其中线性近似中包含有多条线性路线（对应中间状态的不同掩码）。

## 2.3 线性近似表

线性近似表是记录对于给定 $S$ 盒的所有线性近似的偏差信息，其第 $\alpha$ 行第 $\beta$ 列为线性近似掩码为 $(\alpha, \beta)$ 的线性近似值为 0 的输出个数与 $2^{m-1}$ 的差，其中 $m$ 为 S 盒的输入位数。具体来说，对于 $m \times n$ 的 $S$ 盒，线性近似表为一张包含 $2^m$ 行、$2^n$ 列的表，表格中第 $\alpha$ 行，第 $\beta$ 列的元素为

$$\#\{(x, S(x)) \in \mathbb{F}_2^m \times \mathbb{F}_2^n | \alpha \cdot x \oplus \beta \cdot S(x) = 0\} - 2^{m-1}$$

其中 $\alpha \in \{0, 1, ..., 2^m - 1\}, \beta \in \{0, 1, ..., 2^n - 1\}$, 将表中元素除以 $2^m$ 即为相应的线性近似的偏差。

# 3 实验内容

## 3.1 任务一

题目给出的 S 盒输入大小 $m = 4$bit，输出大小 $n = m = 4$bit

所以 S 盒的线性近似表大小就是 $2^4 \times 2^4$, 里面每一项记作 $N(\alpha, \beta)$

先遍历输入掩码 $\alpha \in \mathbf{F}_2^4$，再遍历输出掩码 $\beta \in \mathbf{F}_2^4$，再遍历所有 $2^4$ 种输入 $x$，计算对应线性近似式

$$\alpha \cdot x \oplus \beta \cdot S(x)$$

统计其中为 0 的个数 $N(\alpha, \beta)$，然后在 S 盒的 LAT 中赋值 $N(\alpha, \beta) - 2^3$。

记 parity$(v)$ 为 $v$ 中 1 的个数的奇偶性，& 表示按位与运算

这里为什么要用一个 parity 运算呢，因为实现内积还是挺麻烦的，在我们按位与之后需要把每一位异或起来

不过这里有个 trick，内积的结果不是 0 就是 1，既然如此只要统计有多少个 1 就行了，偶数个 1 就是 0，奇数个 1 就是 1

对应伪代码如算法 1所示。

**算法 1** 4 比特 S 盒线性近似表的计算

**输入：** 4 比特 S 盒 $S[0..15]$

**输出：** 线性近似表 LAT$[0..15, 0..15]$

**for** $\alpha = 0, \ldots, 15$ **do**
  **for** $\beta = 0, \ldots, 15$ **do**
    $N \leftarrow 0$
    **for** $x = 0, \ldots, 15$ **do**
      $p \leftarrow$ parity$(\alpha \& x)$
      $q \leftarrow$ parity$(\beta \& S[x])$
      **if** $p = q$ **then**
        $N \leftarrow N + 1$
      **end if**
    **end for**
    LAT$[\alpha, \beta] \leftarrow N - 8$
  **end for**
**end for**
**return** LAT

代码运行结果如图 1所示。

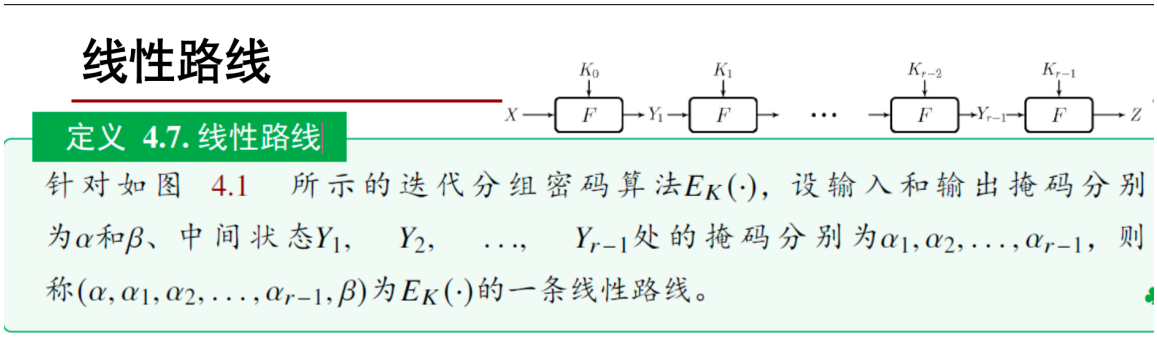
```
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code> uv run python task1.py
        0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
 0 :    8  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
 1 :    0  0  0  0 -2 -2  2  2  0  0  0  0  2  2 -2  6
 2 :    0  0 -4  0  4  0  0  0 -4  0  0  0  0  0 -4  0
 3 :    0 -4  0  0 -2 -2  2 -2  0  0  0  4  2 -2 -2 -2
 4 :    0 -2  2  0  4  2  2  0  4 -2 -2  0  0  2 -2  0
 5 :    0 -2  2  0  2  0 -4  2  0  2  2  4 -2  0  0  2
 6 :    0  2  2  0  0 -2 -2  0  0  2 -6  0  0 -2 -2  0
 7 :    0 -2 -2  0  2 -4  0 -2  0 -2 -2  0 -2  0  4  2
 8 :    0  0  4  0  2 -2  2  2 -4  0  0  0  2  2  2 -2
 9 :    0  4  0  0  0  0  0 -4  0  0  0  4  0  4  0  0
 A :    0  0  0  0 -2 -2  2  2  0  0  0  0 -6  2 -2 -2
 B :    0  0  0 -8  0  0  0  0  0  0  0  0  0  0  0  0
 C :    0  2  2  0  2  0  4 -2  0  2  2  0 -2 -4  0  2
 D :    0 -2 -2  0  0  2  2  0  0  6 -2  0  0  2  2  0
 E :    0 -2  2  0 -2  4  0 -2 -4 -2 -2  0 -2  0  0  2
 F :    0 -2  2  0  0 -2 -2 -4  0  2  2 -4  0  2 -2  0
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code>
```

Figure 1: 任务一计算得到的 S 盒线性近似表

其中偏差最大的就是 $\boxed{0x0B \to 0x03}$

## 3.2 任务二

基于任务一计算得到的 S 盒线性近似表（LAT），尝试找到一条 4 轮迭代线性路线并计算其偏差。




翻遍课本没找到迭代线性路线的定义，不过类比前面迭代差分路线，迭代的思路就是这一轮的输出要作为下一轮的输入，这样可以不断运作下去

所以输入掩码得和输出掩码一样，这样输出掩码又能作为下一轮的输入迭代下去，那 4 轮的迭代线性路线就是形如

$$\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5$$

### 3.2.1 考量: 活跃 S 盒的个数

那这还是一头雾水，主要是 CipherFour 算法有四个 S 盒，而又由于 P 置换这四个 S 盒的比特会相互影响，

看上去一筹莫展，但是我们的堆积引理和偏差的分析，实际上是建立在 Markov 假设上面的，那既然如此

我们总是不希望有太多的变量，那样的话独立性就越来越差，具体来说就是 ppt 这个

- 堆积引理的独立性要求
  - 堆积引理成立的关键在于诸个$X_i$的独立性
  - Matsui对DES的分析中，每一轮的线性近似式只含有1个活跃S盒（输出掩码非零）。若包含相邻的2个以上活跃S盒，利用堆积引理获得最终的线性近似式其偏差值将明显偏离真实值，则密钥信息不再准确

Figure 2: 堆积引理要求

堆积引理用在相关度上就更简洁一点

考虑输入掩码 $\alpha = (\alpha_0, \alpha_1, \alpha_2, \alpha_3)$ 和输出掩码 $\beta = (\beta_0, \beta_1, \beta_2, \beta_3)$,4 个 S 盒对应的一轮 S 层相关度为

$$c(\alpha, \beta) = \prod_{j=0}^{3} c(\alpha_j, \beta_j)$$

因为相关度就是偏差的 2 倍，相关度绝对值越大的路线对于线性分析来说就更好

那我们知道 $c(0,0) = 1$，这就是最大的，那很明显活跃的 S 盒越少越好，相关度越大，当然不能没有，没有活跃 S 盒那就没意义了，就是全 0

但是这也还是有一些问题，也不只有 $c(0,0) = 1$，值得注意的是看前面的 LAT

除了 $0 \to 0$ 是 8 以外，还有一个我们前面找到的 $B \to 3$

```
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code> uv run python task1.py
      0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F
0 :   8  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
1 :   0  0  0  0 -2 -2  2  2  0  0  0  0  2  2 -2  6
2 :   0  0 -4  0  4  0  0  0 -4  0  0  0  0  0 -4  0
3 :   0 -4  0  0 -2 -2  2 -2  0  0  0  4  2 -2 -2 -2
4 :   0 -2  2  0  4  2  2  0  4 -2 -2  0  0  2 -2  0
5 :   0 -2  2  0  2  0 -4  2  0  2  2  4 -2  0  0  2
6 :   0  2  2  0  0 -2 -2  0  0  2 -6  0  0 -2 -2  0
7 :   0 -2 -2  0  2 -4  0 -2  0 -2 -2  0 -2  0  4  2
8 :   0  0  4  0  2 -2  2  2 -4  0  0  0  2  2  2 -2
9 :   0  4  0  0  0  0  0 -4  0  0  0  4  0  4  0  0
A :   0  0  0  0 -2 -2  2  2  0  0  0  0 -6  2 -2 -2
B :   0  0  0 -8  0  0  0  0  0  0  0  0  0  0  0  0
C :   0  2  2  0  2  0  4 -2  0  2  2  0 -2 -4  0  2
D :   0 -2 -2  0  0  2  2  0  0  6 -2  0  0  2  2  0
E :   0 -2  2  0 -2  4  0 -2 -4 -2 -2  0 -2  0  0  2
F :   0 -2  2  0  0 -2 -2 -4  0  2  2 -4  0  2 -2  0
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab3\code>
```

Figure 3: LAT

也就是说 $c(B, 3) = 1$, 这个从相关度来说和 $0 \to 0$ 是一样的, 理论上来说也是可以选用，但是咱们这个是经过 P 层置换的, 引入这个会导致我们的分析变得非常复杂

或许交给 Agent 来说这不过沧海一粟，但人脑的内存还是没有那么大，若无必要勿增实体嘛，那我们还是尽可能走简单的路线，不要给自己找麻烦

既然 0，0 和 B，3 的效果是一样的，那们自然是选择 0,0 来分析

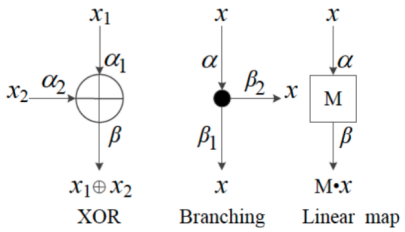
所以说我们只考虑一个活跃 S 盒

### 3.2.2 贪心算法

跟差分路线比起来这个更难了，因为线性变换对掩码是有影响的，

## 线性变换的掩码传播情况——以概率1过线性变换

- 异或运算（XOR）
  - 输入掩码分别为$a_1$、$a_2$，输出掩码为$\beta$，则$a_1 = a_2 = \beta$
- 分支运算（Branching）
  - 输入掩码为$a$，输出掩码分别为$\beta_1$、$\beta_2$，则$\beta_1 \oplus \beta_2 = a$
- 线性映射（Linear Map）
  - 输入掩码为$a$，输出掩码为$\beta$，则$a = M^T\beta$（以AES的矩阵乘为例证明，留作练习）




线性掩码是要经过 P 置换的，不过我们知道置换就是乘以一个矩阵, 在这里不妨就记作 P 吧

这个 P 也比较好求，




那考虑第一轮的输入掩码是 $\alpha_1$, 输出掩码是 $\beta_1$
如果没有这个 P 置换的话，应该是 $\beta_1 = \alpha_2$
但是实际上应该是 $\alpha_2 = P\beta_1$
好吧还是太混沌了，让我们来形式推导一下
最开始的状态是

$$(\alpha_{11}, 0, 0, 0) \xrightarrow{S} (\beta_{11}, 0, 0, 0) \xrightarrow{P} (\alpha_{21}, \alpha_{22}, \alpha_{23}, \alpha_{24})$$

那总而言之就是

$$\alpha_1 \xrightarrow{S} \beta_1 \xrightarrow{P} \alpha_2 \xrightarrow{S} \beta_2 \xrightarrow{P} \alpha_3 \xrightarrow{S} \beta_3 \xrightarrow{P} \alpha_4 \xrightarrow{S} \beta_4 \xrightarrow{P} \alpha_1$$

要写算法肯定要确定性的步骤，我们要找这样一条路线，每一步应该有一个确定性的输出
这里的 P 就是乘矩阵，已经确定了，那 S 本身有很多结果，
不知道该选哪一个，我们肯定是想要偏差尽可能大的，所以我们就选择偏差最大的，那这不是贪心算法嘛？关键是题目只要求我们找到一条路线
贪心算法求出的可能不是全局最优解，但至少也是个局部最优解嘛，也算是不错的
其实我们还可以再贪心一点，我们不是嫌 P 比较麻烦吗，那既然如此我们就找哪个不动点吧。
也就是令 $P\beta = \beta$, 这个其实就是 $(P - I)\beta = 0$, 如果这个矩阵不满秩，那就有非零解
希望它不满秩吧，诶，我们注意到前面的矩阵 P 第一行第一列是 1，那减去单位矩阵之后，第一行就是全 0 了，所以这个矩阵肯定不满秩

所以我们每次就选这样的 $\beta$, 这样就很好了




比如 $\beta = 1000000000000000$, 也就是 $\beta = (8, 0, 0, 0)$

但是再细想一下，迭代型的优点不就是可以不断累积嘛，那就没必要找四轮的，直接找一轮的迭代 4 次不就行了

至此恍然惊醒，那就是

$$\alpha_1 \xrightarrow{S} \beta_1 \xrightarrow{P} \alpha_1$$

$P\beta = \beta$ 还是太保守了，再贪心一点，我们直接让

$$P(\beta_1) = \alpha_1$$

### 3.2.3 意外之喜

这样的好处是，$\beta_1$ 直接被 $\alpha_1$ 确定了，这样我们可以写出非常简洁的算法，

但是有另一个问题，我们前面探讨的几个活跃 S 盒主要是为了方便计算，但是现在我们发现可以只求一轮迭代的路线

那当然也可以从一个活跃 S 盒，2 个活跃 S 盒，开始尝试，但是他就只有 $2^{16} - 1$ 个情况，那其实多少个 S 盒都无所谓了，毕竟就一轮嘛，所以我们直接遍历全部 $2^{16} - 1$ 个非零 $\alpha$，就能包含所有情况，然后可以用 $P^{-1}$ 计算 $\beta$, 从 LAT 中查找各 S 盒的相关度并相乘, 找到最大的相关度，然后重复四轮就行了

对应伪代码如算法 2所示。

**算法 2** CipherFour 四轮迭代线性路线的搜索

**输入：** 4 比特 S 盒线性近似表 LAT$[0..15, 0..15]$，16 比特置换 $P$
**输出：** 四轮迭代线性路线及其理论偏差

$\alpha^* \leftarrow 0,\ \beta^* \leftarrow 0,\ C^* \leftarrow 0$
**for** $\alpha = 1, \ldots, 2^{16} - 1$ **do**
  $\beta \leftarrow P^{-1}(\alpha)$
  $C \leftarrow 1$
  **for** $j = 0, \ldots, 3$ **do**
    $a \leftarrow (\alpha \gg 4j)\&0xF$
    $b \leftarrow (\beta \gg 4j)\&0xF$
    $C \leftarrow C \cdot \mathrm{LAT}[a, b]/8$
  **end for**
  **if** $|C| > |C^*|$ **or** $(|C| = |C^*|$ **and** $\alpha > \alpha^*)$ **then**
    $\alpha^* \leftarrow \alpha$
    $\beta^* \leftarrow \beta$
    $C^* \leftarrow C$
  **end if**
**end for**
$C_4 \leftarrow (C^*)^4$
$\varepsilon_4 \leftarrow C_4/2$
$T \leftarrow [\alpha^*, \alpha^*, \alpha^*, \alpha^*]$
**return** $T, \beta^*, \varepsilon_4$

代码运行结果如图 4所示。

```
wjy@wjydeMacBook-Air crypto-analyze % /usr/local/bin/python3 /Users/wjy/大三/实验
================================================================================
任务二：CipherFour 四轮迭代线性路线搜索
================================================================================
S盒（实验题目表1.1）：9 8 0 1 E B C D 3 2 5 7 F A 6 4
P置换（课程参考程序）：[0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15]
非零相关度的单轮固定点候选数量：3152

搜索得到的一条最优单轮固定点迭代路线：
  S层输入掩码 alpha = 0x8000
  S层输出掩码 beta  = 0x8000
  P(beta)           = 0x8000
  满足 P(beta) = alpha: True

逐轮信息（4轮，每轮重复同一组S层掩码）：
  第1轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)
  第2轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)
  第3轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)
  第4轮：0x8000 --S--> 0x8000 --P--> 0x8000; C = -1/2 (-0.50000000)

4轮输入掩码路线：
  0x8000 -> 0x8000 -> 0x8000 -> 0x8000 -> 0x8000

每个S盒的LAT贡献：
  S1: (a,b)=(8,8), LAT[8][8]=-4, C=-1/2
  S2: (a,b)=(0,0), LAT[0][0]=8, C=1
  S3: (a,b)=(0,0), LAT[0][0]=8, C=1
  S4: (a,b)=(0,0), LAT[0][0]=8, C=1

单轮相关度 C1      = -1/2 = -0.50000000
单轮偏差 epsilon1   = -1/4 = -0.25000000
4轮路线相关度 C4    = 1/16 = +0.06250000
4轮路线理论偏差     = 1/32 = +0.03125000
4轮路线绝对偏差     = 1/32 = 0.03125000
注：轮密钥可能改变偏差符号；此处给出不含密钥符号因子的路线偏差。
注：单条路线的偏差不等于线性包络的总偏差。
wjy@wjydeMacBook-Air crypto-analyze %
```

Figure 4: 任务二四轮迭代线性路线搜索结果

程序共找到 3152 个单轮相关度非零的候选，其中一条绝对相关度最大的路线满足

$$\alpha = 0x8000, \qquad \beta = 0x8000, \qquad P(0x8000) = 0x8000.$$

因此，该路线可迭代 4 轮，得到

$$0x8000 \longrightarrow 0x8000 \longrightarrow 0x8000 \longrightarrow 0x8000 \longrightarrow 0x8000.$$

对于每一轮，仅第一个 S 盒的输入、输出掩码非零，分别为 $(8,8)$；其余 3 个 S 盒的掩码均为 $(0,0)$。由任务一所得 LAT 可知

$$\mathrm{LAT}[8,8] = -4, \qquad \mathrm{LAT}[0,0] = 8.$$

故单轮相关度为

$$C_1 = \frac{-4}{8}\left(\frac{8}{8}\right)^3 = -\frac{1}{2},$$

相应的单轮偏差为 $\varepsilon_1 = C_1/2 = -1/4$。根据堆积引理（Piling-up Lemma），4 轮**单条线性路线**的相关度为

$$C_4 = \prod_{i=1}^{4} C_i = \left(-\frac{1}{2}\right)^4 = \frac{1}{16}.$$

从而得到不计密钥常数项时的理论偏差

$$\varepsilon_4 = \frac{C_4}{2} = \frac{1}{32} = 0.03125.$$

但是我们这个还是投机取巧了，我们好奇这个是不是最好的，这个也就算是一个贪心解，但是你如果考虑多轮迭代路线，比如 2 个 2 轮拼在一起，或者单个 4 轮的

那这个未必就是最好的，但是你考虑两轮、四轮的话，这个活跃 S 盒又成了比较大的问题，因为这时候咱们不能通过 P 置换直接计算中间状态了

那这种问题我们尝试问一下 AI



用时 4m 55s

全局最优四轮闭环是

$$8000 \to 8000 \to 8000 \to 8000 \to 8000.$$

按最高位为第 0 位编号，$P[i]$ 表示把源位 $i$ 移到目标位 $P[i]$；例如 $P(1234) = 016A$，且 $P(8000) = 8000$。下表采用完整 Walsh LAT，相关度为 LAT 除以 16。

| 轮次 | 输入掩码 | S 盒输出掩码 | P 后掩码 | 四个 S 盒的 LAT（高位到低位） | 本轮相关度 |
|---|---|---|---|---|---|
| 1 | 8000 | 8000 | 8000 | $(-8, 16, 16, 16)$ | $-1/2$ |
| 2 | 8000 | 8000 | 8000 | $(-8, 16, 16, 16)$ | $-1/2$ |
| 3 | 8000 | 8000 | 8000 | $(-8, 16, 16, 16)$ | $-1/2$ |
| 4 | 8000 | 8000 | 8000 | $(-8, 16, 16, 16)$ | $-1/2$ |

因此路线相关度为 $C = (-1/2)^4 = 1/16$，偏差绝对值为 $|\varepsilon| = |C|/2 = 1/32$。若沿用前文的半 Walsh LAT 约定，表中每项除以 2，即活跃 S 盒的 `LAT[8][8] = -4`。

完整的标准库 Python 验证脚本 对全部 65,535 个非零起始掩码计算四轮开放路线的精确上界。只有 `00BB`、`0660`、`0BB0`、`B00B`、`B0B0`、`B0BB`、`BB0B` 七个起点的上界可能超过 $1/16$；脚本继续检查它们所有可能超越该值的闭环，结果均未超越。搜索允许任意数量的活跃 S 盒，也没有禁止中途重复状态，因此 $1/32$ 是所求的全局最大偏差绝对值。

星期日16:16

值得注意的是，这个求出来的还真就是最好的，也算是意外之喜了

## 3.3 任务三

题目给出的 PHILOX-2W 算法是一种基于 Feistel 结构的分组密码，其左右分支长度均为 $W = 8$bit，乘法常数为 $M = 0x9B$。实验要求分析输入掩码分别为 $(0, 1)$、$(1, 0)$、$(0, 2)$、$(2, 1)$ 时的一轮线性掩码传播情况，并找出每组输入掩码对应的输出偏差绝对值最大的前 5 个输出掩码及其偏差。

根据题目给出的轮函数，设输入状态为 $(L, R)$，轮密钥为 $K$，一轮输出状态为 $(L', R')$，则有

$$L' = \text{mullo}_W(R),$$
$$R' = \text{mulhi}_W(R) \oplus L \oplus K,$$

其中

$$\text{mulhi}_W(R) = \left\lfloor \frac{R \cdot M}{2^W} \right\rfloor, \qquad \text{mullo}_W(R) = (R \cdot M) \bmod 2^W.$$

设输入掩码为 $\alpha = (\alpha_L, \alpha_R)$，输出掩码为 $\beta = (\beta_L, \beta_R)$，则对应的线性近似式为

$$\alpha_L \cdot L \oplus \alpha_R \cdot R \oplus \beta_L \cdot L' \oplus \beta_R \cdot R'.$$

将轮函数代入上述表达式，整理可得

$$(\alpha_L \oplus \beta_R) \cdot L \oplus \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \beta_R \cdot \text{mulhi}_W(R) \oplus \beta_R \cdot K.$$

现在把 L 和 R 分开的，后面的部分只有 R, 那其实可以写成

$$(\alpha_L \oplus \beta_R) \cdot L \oplus f(R, K)$$

因为 $L, R$ 是独立的，这样写的好处是我们可以分别讨论了
输入分支 $L$ 在 $\mathbf{F}_2^8$ 上均匀分布，因为输入是随机的嘛

### 3.3.1 $\alpha_L \neq \beta_R$

那么显然

$$\Pr((\alpha_L \oplus \beta_R) \cdot L = 0) = \Pr((\alpha_L \oplus \beta_R) \cdot L = 1) = \frac{1}{2}$$

然后

$$\Pr(f(R, K) = 0) = \Pr(f(R, K) = 1) = \frac{1}{2}$$

那么根据堆积引理，最后 $\Pr((\alpha_L \oplus \beta_R) \cdot L \oplus f(R, K) = 0) = \frac{1}{2}$
也就是说偏差都为 0，这种情况是平凡的

### 3.3.2 $\alpha_L = \beta_R$

这个时候我们发现线性近似式和 L 无关了

$$\alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \alpha_L \cdot \text{mulhi}_W(R) \oplus \alpha_L \cdot K.$$

但是这里还有一个 $K$ 啊，不过 $K$ 是固定的，我们做线性近似表的时候，只有输入在变
那所以说 $\alpha_L \cdot K$ 这个在线性近似表的每一项中都是固定的值，要么是 0，要么是 1
那这其实不会影响线性偏差的绝对值，如果 $\alpha_L \cdot K = 0$,
那线性近似式等于 0 的输入的个数，就相当于使得前面这个式子为 0 的输入的个数
也就是说

$$\text{LAT}(\alpha, \beta) = \#\{R | \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \alpha_L \cdot \text{mulhi}_W(R) = 0\} - 2^7$$

如果 $\alpha_L \cdot K = 1$,
那线性近似式等于 0 的输入的个数，就相当于使得前面这个式子为 1 的输入的个数, 那就是拿总数 $2^8$ 去减嘛
也就是说

$$\text{LAT}(\alpha, \beta) = 2^8 - \#\{R | \alpha_R \cdot R \oplus \beta_L \cdot \text{mullo}_W(R) \oplus \alpha_L \cdot \text{mulhi}_W(R) = 0\} - 2^7$$

也就是

$$\mathrm{LAT}(\alpha,\beta)=2^7-\#\{R|\alpha_R\cdot R\oplus\beta_L\cdot\mathrm{mullo}_W(R)\oplus\alpha_L\cdot\mathrm{mulhi}_W(R)=0\}$$

这和前面就是加个负号的区别，绝对值是一样的，那为了方便起见，我们就令 $\alpha_L\cdot K=0$

先遍历题目给出的 4 组输入掩码 $\alpha=(\alpha_L,\alpha_R)$，根据上述条件确定输出掩码右分支 $\beta_R=\alpha_L$，再遍历全部 $2^8$ 种输出左分支掩码 $\beta_L$。对于每组输入、输出掩码，遍历全部 $2^8$ 种右分支输入 $R$，计算对应线性近似式是否为 0。统计其中为 0 的个数 $N(\alpha,\beta)$，并计算线性偏差

$$\varepsilon(\alpha,\beta)=\frac{N(\alpha,\beta)}{2^8}-\frac{1}{2}.$$

最后将所有输出掩码按照线性偏差的绝对值降序排列，选取前 5 个输出掩码及其偏差。

记 parity$(v)$ 为 $v$ 中 1 的个数的奇偶性，& 表示按位与运算

对应伪代码如算法 3所示。

**算法 3** PHILOX-2W 一轮线性掩码传播分析

**输入：** $W=8$，$M=0x9B$，输入掩码集合 $\mathcal{A}=\{(0,1),(1,0),(0,2),(2,1)\}$

**输出：** 各输入掩码对应的前 5 个输出掩码及其线性偏差

**for** $(\alpha_L,\alpha_R)\in\mathcal{A}$ **do**
  $T\leftarrow\emptyset$
  $\beta_R\leftarrow\alpha_L$
  **for** $\beta_L=0,\ldots,255$ **do**
    **if** $(\beta_L,\beta_R)=(0,0)$ **then**
      **continue**
    **end if**
    $N\leftarrow 0$
    **for** $R=0,\ldots,255$ **do**
      $H\leftarrow\lfloor R\cdot M/2^8\rfloor$
      $D\leftarrow(R\cdot M)\bmod 2^8$
      $p\leftarrow\mathrm{parity}(\alpha_R\,\&R)$
      $q\leftarrow\mathrm{parity}(\beta_L\&D)\oplus\mathrm{parity}(\beta_R\wedge H)$
      **if** $p=q$ **then**
        $N\leftarrow N+1$
      **end if**
    **end for**
    $\varepsilon\leftarrow N/256-1/2$
    $T\leftarrow T\cup\{(\beta_L,\beta_R,\varepsilon)\}$
  **end for**
  按 $|\varepsilon|$ 降序排列 $T$
  Result$[\alpha]\leftarrow$ Top5$(T)$
**end for**
**return** Result

代码运行结果如图 5所示。

```
wjy@wjydeMacBook-Air crypto-analyze % /usr/local/bin/python3 /Users/wjy/大三/实
================================================================================
PHILOX-2W 一轮线性掩码传播分析
================================================================================
W = 8
M = 0x9B
K = 0x00

--------------------------------------------------------------------------------
输入掩码 alpha = (0x00, 0x01)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0x01, 0x00)      +0.50000000       0.50000000
2       (0x02, 0x00)      +0.00000000       0.00000000
3       (0x03, 0x00)      +0.00000000       0.00000000
4       (0x04, 0x00)      +0.00000000       0.00000000
5       (0x05, 0x00)      +0.00000000       0.00000000

--------------------------------------------------------------------------------
输入掩码 alpha = (0x01, 0x00)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0x5A, 0x01)      -0.14453125       0.14453125
2       (0xDA, 0x01)      +0.11328125       0.11328125
3       (0xDB, 0x01)      +0.11328125       0.11328125
4       (0xD2, 0x01)      +0.10546875       0.10546875
5       (0x5B, 0x01)      +0.08984375       0.08984375

--------------------------------------------------------------------------------
输入掩码 alpha = (0x00, 0x02)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0x03, 0x00)      +0.50000000       0.50000000
2       (0x01, 0x00)      +0.00000000       0.00000000
3       (0x02, 0x00)      +0.00000000       0.00000000
4       (0x04, 0x00)      +0.00000000       0.00000000
5       (0x05, 0x00)      +0.00000000       0.00000000

--------------------------------------------------------------------------------
输入掩码 alpha = (0x02, 0x01)
--------------------------------------------------------------------------------
Rank    Output Mask          Bias             Absolute Bias
1       (0xB6, 0x02)      +0.17578125       0.17578125
2       (0xA4, 0x02)      +0.16015625       0.16015625
3       (0xF4, 0x02)      -0.13671875       0.13671875
4       (0xB4, 0x02)      -0.11328125       0.11328125
5       (0xBC, 0x02)      -0.10546875       0.10546875

================================================================================
全部输入掩码分析完成!
```

Figure 5: PHILOX-2W 一轮线性掩码传播分析结果

结果如下

Table 1: PHILOX-2W 一轮线性掩码传播分析结果

| 输入掩码 $\alpha$ | Rank | 输出掩码 $\beta$ | Bias | Absolute Bias |
|---|---|---|---|---|
| $(0x00, 0x01)$ | 1 | $(0x01, 0x00)$ | $+0.50000000$ | $0.50000000$ |
| | 2 | $(0x02, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| | 3 | $(0x03, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| | 4 | $(0x04, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| | 5 | $(0x05, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| $(0x01, 0x00)$ | 1 | $(0x5A, 0x01)$ | $-0.14453125$ | $0.14453125$ |
| | 2 | $(0xDA, 0x01)$ | $+0.11328125$ | $0.11328125$ |
| | 3 | $(0xDB, 0x01)$ | $+0.11328125$ | $0.11328125$ |
| | 4 | $(0xD2, 0x01)$ | $+0.10546875$ | $0.10546875$ |
| | 5 | $(0x5B, 0x01)$ | $+0.08984375$ | $0.08984375$ |
| $(0x00, 0x02)$ | 1 | $(0x03, 0x00)$ | $+0.50000000$ | $0.50000000$ |
| | 2 | $(0x01, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| | 3 | $(0x02, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| | 4 | $(0x04, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| | 5 | $(0x05, 0x00)$ | $+0.00000000$ | $0.00000000$ |
| $(0x02, 0x01)$ | 1 | $(0xB6, 0x02)$ | $+0.17578125$ | $0.17578125$ |
| | 2 | $(0xA4, 0x02)$ | $+0.16015625$ | $0.16015625$ |
| | 3 | $(0xF4, 0x02)$ | $-0.13671875$ | $0.13671875$ |
| | 4 | $(0xB4, 0x02)$ | $-0.11328125$ | $0.11328125$ |
| | 5 | $(0xBC, 0x02)$ | $-0.10546875$ | $0.10546875$ |

值得注意的是，偏差为 0.5, 意味着就是必然事件了，那这就是一个非常好的区分器

# 4 实验报告完整地址

实验完整代码、实验报告 LaTeX 源代码见仓库：https://github.com/infinitepwn/crypto-analyze.git

# 5 AI 使用情况

本实验使用 AI 辅助编程和实验报告的排版、错别字检查，实验主体流程设计、结果分析和实验报告的撰写未使用 AI。